

สรุป

ถ้าไม่มี connector ทั้งคู่ จะใช้ TASKS.md อย่างเดียว
ได้อย่างไร

ได้ครับ ใช้ TASKS.md อย่างเดียวได้เลย โดยไม่ต้อง
มี Jira หรือ Asana เพราะ skill นี้ออกแบบมาให้จัดการ
งานผ่านไฟล์ Markdown กลางที่ทั้งคุณและ Agent
อ่าน/แก้ไขร่วมกันได้[1]

1. สร้างไฟล์ TASKS.md

วางไฟล์ไว้ใน โฟลเดอร์งานปัจจุบันของ FIG Agent
แล้วใส่โครงสร้างมาตรฐานนี้:

```
markdown
```



```
# Tasks
```

Active

Waiting On

Someday

Done

ถ้าเรียกใช้ productivity start skill ได้ ให้ส่ง:

text



เริ่มระบบ productivity โดยใช้ TASKS.md
เป็นแหล่งเก็บ task หลัก

ระบบจะตรวจว่า TASKS.md มีอยู่หรือไม่ และสร้าง
ไฟล์ให้ถ้ายังไม่มี[2]

2. รูปแบบ task

ใช้ checkbox และเขียนข้อมูลสำคัญไว้ในบรรทัดเดียว:

markdown



Active

- [] ****ตรวจสอบ Supabase schema**** - ตรวจสอบ tables, columns, indexes และ RLS; due 2026-09-11
- [] ****เตรียม FIG Knowledge**** - เพิ่มกฎให้ Agent ตรวจสอบ migration ก่อนแก้ database; due 2026-09-12

Waiting On

- [] ****รอ approval write grant**** - สำหรับ branch `chore/issue-templates`; since 2026-09-10

Someday

- [] **เชื่อมต่อ Jira หรือ Asana** - ทำเมื่อมี connector

Done

- [x] ~~สร้างแนวทาง Supabase schema awareness~~ (2026-09-10)

หลักที่ควรใช้:

- Active — กำลังทำหรือต้องทำเร็ว ๆ นี้
- Waiting On — รอคนอื่น ข้อมูล หรือ approval
- Someday — ยังไม่เร่งด่วน
- Done — เสร็จแล้ว
- ใช้ **ตัวหนา** กับชื่อ task
- ใช้ due สำหรับกำหนดส่ง
- ใช้ since สำหรับงานที่กำลังรอ

- งานที่เสร็จแล้วให้ใช้ checkbox [x] และขีดฆ่าชื่อ task[1]

3. สร้างงาน Agent

หลังสร้างไฟล์แล้ว ใช้คำสั่งธรรมชาติได้เลย:

text



แสดง task ที่กำลังทำอยู่

text



เพิ่ม task: ตรวจสอบ Supabase migration
ภายในพรุ่งนี้

text



ย้าย task "ตรวจสอบ Supabase schema"
ไป Waiting On

text



ทำเครื่องหมายว่า "เตรียม FIG Knowledge"
เสร็จแล้ว

text



สรุป task ที่ค้างและงานที่เลยกำหนด

text



ดึง action items จากบทสนทนานี้ แต่แสดง

ให้ฉันยืนยันก่อนเพิ่มลง TASKS.md

สำหรับคำสั่งที่เพิ่มหรือแก้ไขไฟล์ แนะนำให้ตั้งกฎว่า Agent ต้องแสดง diff ก่อนบันทึกจริง

4. เพิ่มกฎป้องกัน task ซ้ำ

ใส่คำสั่งนี้ไว้ใน FIG Knowledge:

text



สำหรับ task-management ให้ใช้ TASKS.md ในโฟลเดอร์งานปัจจุบันเป็นแหล่งข้อมูลหลัก

ก่อนเพิ่ม task ใหม่:

1. อ่าน TASKS.md ก่อน
2. ตรวจสอบ งานที่เกี่ยวข้อง และ due date ว่าซ้ำกับ task เดิมหรือไม่
3. ถ้าซ้ำ ให้ปรับปรุง task เดิมแทนการสร้าง

รายการใหม่

4. ถ้าไม่ซ้ำ ให้แสดง task ที่จะเพิ่มและขอการยืนยัน

5. ห้ามลบ task โดยไม่ขอการยืนยัน

6. เมื่อย้ายหรือทำ task เสร็จ ให้บันทึกวันที่และบริบทสั้น ๆ

5. ใช้ร่วมกับ daily briefing

แม้ไม่มี calendar, Jira หรือ Asana ก็ยังใช้ได้:

text



ทำ daily briefing จาก TASKS.md อย่างเดียว

แสดง Active, Waiting On, งานที่ due วันนี้ และงานที่ค้างเกินกำหนด

text



สร้าง standup จากการเปลี่ยนแปลงใน
TASKS.md
จัดเป็น เมื่อวาน / วันนี้ / Blockers

ข้อจำกัดคือ TASKS.md เป็นรายการงานแบบ static
ไม่ใช่ระบบแจ้งเตือนตามเวลาโดยอัตโนมัติ ดังนั้น
due จะช่วยให้ Agent ตรวจสอบและรายงานงานที่ถึง
กำหนด แต่ไม่ได้ทำให้เกิด notification เอง[3]

โครงสร้างที่แนะนำสำหรับคุณ

text



```
project/  
├── TASKS.md  
├── FIG-KNOWLEDGE.md
```

ให้กำหนดชัดเจนว่า:

text



TASKS.md คือ source of truth สำหรับงานทั้งหมด

FIG-KNOWLEDGE.md คือกฎและบริบทการทำงานของ Agent

ห้าม Agent แก้ FIG-KNOWLEDGE.md จากการเพิ่ม task

ห้าม Agent สร้าง task ซ้ำ

ทุกการเปลี่ยนแปลง TASKS.md ต้องสรุปให้ผู้ใช้ทราบ

ดังนั้นคุณสามารถเริ่มใช้ task-management ได้ทันทีโดยไม่ต้องเชื่อมต่อ Jira หรือ Asana ครับ.

การอ้างอิง:

- [1] task-management skill by anthropics/
knowledge-work-plugins <https://playbooks.com/skills/anthropics/knowledge-work-plugins/task-management>
- [2] Start Claude Skill for Productivity by
anthropics | mcpbeat <https://mcpbeat.com/skills/anthropics/knowledge-work-plugins-productivity-start/>
- [3] task-management skill: 'remind me to' creates
a static task, not a time ... <https://github.com/anthropics/knowledge-work-plugins/issues/141>
- [4] Computer-Use https://github.com/asgeirtj/system_prompts_leaks/blob/main/Anthropic/claude-cowork.md
- [5] knowledge-work-plugins/operations/
README.md at main - GitHub <https://github.com/anthropics/knowledge-work-plugins/blob/main/operations/README.md>
- [6] Anthropic Knowledge Work Plugins: Open

Source Plugins for ... <https://pyshine.com/Anthropic-Knowledge-Work-Plugins-Claude-Cowork/>

[7] task-management - skills <https://explainx.ai/skills/anthropics/knowledge-work-plugins/task-management>

[8] productivity <https://www.claudepluginhub.com/plugins/fuww-productivity-productivity>

[9] task-management | Skills Marketplace - LobeHub <https://lobehub.com/skills/anthropics-knowledge-work-plugins-task-management>

[10] anthropics/knowledge-work-plugins - Skills <https://explainx.ai/skills/anthropics/knowledge-work-plugins>

วิธีให้ Agent ช่วยจัดลำดับความสำคัญใน TASKS.md

ให้ Agent จัดลำดับความสำคัญใน TASKS.md ได้โดยเพิ่มกติกา priority และให้ Agent วิเคราะห์ก่อนแก้

ไฟล์วิธีนี้ยังใช้ได้แม้ไม่มี Jira หรือ Asana เพราะ TASKS.md เป็นแหล่งข้อมูลหลักของคุณ[1]

1. เพิ่ม priority ให้แต่ละ task

ใช้ระดับง่าย ๆ:

- P0 — เร่งด่วนมาก ต้องทำทันที
- P1 — สำคัญ ควรทำวันนี้หรือเร็วที่สุด
- P2 — สำคัญแต่รอได้ วางแผนทำภายในสัปดาห์
- P3 — ยังไม่เร่งด่วน เก็บไว้ทำภายหลัง

ตัวอย่าง:

markdown



Tasks

Active

- [] ****ตรวจสอบ Supabase schema**** -
ตรวจสอบ tables, columns และ RLS; priority:
P0; due: 2026-09-11
- [] ****เขียน schema inventory**** - สรุป
ตารางที่มีอยู่; priority: P1; due: 2026-09-12
- [] ****ปรับปรุง FIG Knowledge**** - เพิ่มกฎ
ตรวจสอบ migration; priority: P2
- [] ****ทดลองสร้าง dashboard**** - priority:
P3

Waiting On

- [] ****รอ approval write grant**** - branch
`chore/issue-templates`; priority: P1;
since: 2026-09-10

Someday

Done

2. เพิ่มกฎใน FIG Knowledge

วางข้อความนี้ไว้ใน Knowledge:

text



Task prioritization

ใช้ TASKS.md เป็น source of truth สำหรับ task ทั้งหมด

เมื่อผู้ใช้ถามเรื่องลำดับความสำคัญ:

1. อ่าน TASKS.md ก่อน
2. ตรวจสอบ priority, due date, dependencies, blockers และ effort
3. จัดลำดับตามกฎ:
 - P0 ก่อน P1 ก่อน P2 ก่อน P3
 - งานที่มี deadline ใกล้กว่ามาก่อน
 - งานที่ block งานอื่นให้มาก่อนงานที่ไม่ block
 - งานที่รอ approval หรือข้อมูลให้แสดงใน Waiting On

- ถ้าไม่มี deadline ให้พิจารณาผลกระทบ และ effort

4. เสนอ Top 1-3 tasks ของวันนี้

5. ห้ามเปลี่ยน priority หรือย้าย task โดยไม่ขอการยืนยัน

6. ก่อนแก้ TASKS.md ให้แสดง proposed changes เป็นรายการสั้น ๆ

การให้ Agent พิจารณา dependency, deadline และ blocker ช่วยให้ลำดับไม่ใช่แค่เรียงตามความรู้สึก โดยเฉพาะงานที่เป็นคอขวดของงานอื่น[2]

3. ใช้คำสั่งจัดลำดับ

ลองใช้คำสั่งเหล่านี้:

text



อ่าน TASKS.md แล้วจัดลำดับงานทั้งหมด จากสำคัญที่สุดไปน้อยที่สุด

text



จาก TASKS.md เลือก 3 งานที่ควรทำวันนี้
พร้อมเหตุผล

text



วิเคราะห์ว่า task ไหน block งานอื่น และควร
ทำอะไรก่อน

text



ตรวจ task ที่ใกล้ถึงกำหนดหรือเลยกำหนด

text



เสนอ priority P0-P3 ให้ทุก task แต่ยังไม่
ต้องแก้ไฟล์

ถ้าคุณอนุมัติแล้วจึงส่ง:

text



ยืนยันการจัด priority ตามข้อเสนอ และ
อัปเดต TASKS.md

4. ให้ Agent คำนวณคะแนน

หากต้องการการจัดลำดับที่สม่ำเสมอ ให้ใช้คะแนน 1–5:

ให้คะแนนทุก task:

- Impact: ผลกระทบถ้าไม่ทำ
- Urgency: ความเร่งด่วนหรือ deadline
- Dependency: จำนวนงานที่ถูก block
- Effort: ความยาก โดยงานที่ใช้แรงน้อยได้คะแนนสูงกว่า

คำนวณ:

$$\text{Priority score} = (\text{Impact} + \text{Urgency} + \text{Dependency}) \times (6 - \text{Effort})$$

แสดงคะแนน เหตุผล และลำดับที่แนะนำก่อน
แก้ TASKS.md

แนวคิดการรวม impact, risk และ effort เป็นสูตร
คะแนนมีใช้ใน skill ด้าน technical debt เช่นกัน โดย
ให้งานที่ผลกระทบและความเสี่ยงสูง แต่ใช้ effort ต่ำ
ได้ลำดับสูงขึ้น[3]

5. รูปแบบรายงานประจำวันที่น่าสนใจ

text



จัดลำดับงานวันนี้จาก TASKS.md โดยแสดง:

1. Top priority 1-3 งาน
2. เหตุผลที่เลือก
3. งานที่ block งานอื่น
4. งานที่รอกคนอื่น
5. งานที่ควรเลื่อนหรือย้ายไป Someday
6. การเปลี่ยนแปลงที่เสนอ โดยยังไม่แก้ไฟล์

ตัวอย่างผลลัพธ์:

text



Top priorities วันนี้

1. P0 ตรวจสอบ Supabase schema

เหตุผล: เป็น prerequisite ก่อนงาน database อื่นทั้งหมด

2. P1 ติดตาม approval write grant

เหตุผล: เป็น blocker ของ issue templates

3. P1 เขียน schema inventory

เหตุผล: ช่วยป้องกันการสร้างตารางซ้ำ

แนะนำให้ย้าย "ทดลองสร้าง dashboard" ไป Someday
ยังไม่ได้แก้ TASKS.md

แนะนำให้จำกัดงานหลักของแต่ละวันไว้ประมาณ 1-3 งาน เพื่อไม่ให้ Agent เลือกงานมากเกินไปกว่าที่ทำได้จริง และให้ตรวจทบทวนรายการทุกเช้าหรือเมื่อมี deadline ใหม่ครับ.

การอ้างอิง:

[1] task-management — AI Skill <https://>

pluginbench.com/skills/anthropics/knowledge-work-plugins/task-management

[2] TaskManager: Complete Digital FTE

Capstone | Agent Factory [https://](https://agentfactory.panaversity.org/docs/Building-Custom-Agents/anthropic-agents-kit-development/taskmanager-capstone)

agentfactory.panaversity.org/docs/Building-Custom-Agents/anthropic-agents-kit-development/taskmanager-capstone

[3] anthropics/knowledge-work-plugins - tech-debt - GitHub <https://github.com/anthropics/knowledge-work-plugins/blob/main/engineering/skills/tech-debt/SKILL.md>

[4] Spec-Driven ADK Agent Development with Antigravity and Spec-kit [https://](https://codelabs.developers.google.com/sdd-adk-antigravity)

codelabs.developers.google.com/sdd-adk-antigravity

[5] GitHub - tasksmd/tasks.md: A lightweight spec for AI agent task queues [https://](https://github.com/tasksmd/tasks.md)

github.com/tasksmd/tasks.md

[6] antopolskiy/kanban-md <https://github.com/antopolskiy/kanban-md>

[7] Build Your AI Agent's Operating System | A.I. Maniacs <https://ai-maniacs.com/docs/playbooks/intermediate/ai-agent-operating-system>

[8] Task Management Skill for Efficient Task Tracking | Get Claude Skills <https://www.getclaudeskills.com/skills/task-management-anthropics>

[9] Agent Skills: Project Management <https://agent-skills.md/skills/JK-0001/skills/project-management>

[10] productivity - Claude Code Plugin <https://www.claudepluginhub.com/plugins/anthropics-productivity-productivity>

[11] update | productivity | ClaudePluginHub <https://www.claudepluginhub.com/skills/anthropics-productivity-productivity/update>

วิธีเขียน Prompt ให้ Agent ดึงงาน P0 ไปทำอัตโนมัติ

ได้ครับ ให้เขียน Prompt เป็นคำสั่งวนรอบที่ชัดเจน:
อ่าน → เลือก P0 → ตรวจ blocker → claim → ทำ →
ทดสอบ → อัปเดตสถานะ → รายงาน แนวทาง
TASKS.md ที่รองรับ agent queue ใช้การ claim งาน
ด้วยรหัส Agent เพื่อป้องกันหลาย Agent หยิบงาน
เดียวกัน[1]

Prompt พร้อมใช้

ใส่ Prompt นี้ใน FIG Knowledge หรือใช้เป็นคำสั่งเริ่ม
งาน:

text



คุณเป็น Autonomous Task Agent ที่ทำงาน
จาก TASKS.md

เมื่อได้รับคำสั่ง "ทำงานต่อ" ให้ทำตามลำดับ
นี้:

1. อ่าน TASKS.md ก่อนทุกครั้ง
2. ค้นหา task ที่ยังไม่เสร็จและมี priority สูงสุด
3. ให้เลือก P0 ก่อน P1, P1 ก่อน P2 และ P2 ก่อน P3
4. ห้าม task ที่มีสถานะ blocked, Waiting On หรือมี dependency ที่ยังไม่เสร็จ
5. ถ้ามีหลาย task ระดับเดียวกัน ให้เลือก task ที่:
 - unblock งานอื่นได้มากที่สุด
 - deadline ใกล้ที่สุด
 - acceptance criteria ชัดเจนที่สุด
6. ก่อนเริ่มทำ ให้ claim task โดยเพิ่ม: (@fig-agent)
7. ห้าม Agent อื่นหรือ session อื่นหยิบ task ที่มีผู้ claim แล้ว
8. อ่านรายละเอียด, files, acceptance criteria และ blocked by ให้ครบ
9. ทำงานเฉพาะ task ที่ claim แล้ว
10. ทดสอบหรือ validate ผลลัพธ์ก่อนรายงานว่าเป็นเสร็จ
11. เมื่อเสร็จ ให้เปลี่ยน checkbox เป็น [x] และย้ายไปส่วน Done
12. ถ้าทำไม่ได้ ให้ย้ายไป Waiting On หรือเพิ่ม blocker พร้อมเหตุผล

13. ถ้าต้องลบไฟล์, แก้ข้อมูลจริง, deploy, migration, หรือทำ destructive action ให้หยุดและขอ approval

14. รายงาน task ที่ทำ, ไฟล์ที่เปลี่ยน, ผลการทดสอบ และ task ถัดไปที่แนะนำ

รูปแบบ TASKS.md

เพื่อให้ Agent ดึง P0 ได้แม่นยำ ควรแยกหัวข้อตาม priority:

markdown



Tasks

P0

- [] ตรวจสอบ Supabase schema ก่อนแก้ไข database
 - ID: supabase-schema-check

- Details: ตรวจ tables, columns, indexes และ RLS

- Acceptance: มี schema inventory และ ยืนยันว่าไม่มี table ซ้ำ

- Files: FIG-KNOWLEDGE.md

P1

- [] เตรียม migration สำหรับ index

- ID: add-index

- Blocked by: supabase-schema-check

P2

- [] ปรับปรุงเอกสาร workflow

P3

- [] ทดลองทำ dashboard

Waiting On

- [] รอ approval write grant

- ID: issue-template-approval

Done

โครงสร้าง P0–P3 และ metadata เช่น ID, details, files, acceptance และ blocked-by ช่วยให้ Agent เลือกงานที่พร้อมทำและตรวจผลลัพธ์ได้เป็นระบบ[2]

วิธีเรียกใช้

ใช้คำสั่งสั้น ๆ:

text



ทำงานต่อจาก TASKS.md

หรือ:

text



เลือกและทำ task P0 ที่พร้อมทำหนึ่งงาน

ถ้าต้องการให้วนต่อหลังจากแรกเสร็จ:

text



ทำงานจาก TASKS.md ต่อเนื่อง:

- ทำทีละ task
- เลือก P0 ที่ unblocked ก่อนเสมอ
- หลังจบแต่ละ task ให้อัปเดตไฟล์
- หยุดเมื่อไม่มี task ที่พร้อมทำ หรือเจอสิ่งที่ต้อง approval
- ห้ามทำ destructive action เอง

มาตรฐาน TASKS.md บางแบบใช้คำสั่ง /next-task เพื่อค้นหา task ที่มี priority สูงสุด ตรวจสอบ dependency แล้ว claim งานก่อนเริ่มทำ[1]

กฎความปลอดภัย

อย่าใช้คำว่า “ทำทุกอย่างอัตโนมัติ” โดยไม่มีขอบเขต เพราะ Agent อาจแก้ไขไฟล์หรือข้อมูลจริงเกินกว่าที่ต้องการ ให้เพิ่มกฎนี้:

text



อนุญาตอัตโนมัติ:

- อ่านไฟล์
- วิเคราะห์ task
- แก้เอกสาร
- เพิ่มผลการตรวจสอบ
- รัน test ที่ไม่เปลี่ยนข้อมูลจริง

ต้องขอ approval ก่อน:

- ลบหรือเขียนทับไฟล์สำคัญ
- แก้ฐานข้อมูล Supabase
- รัน migration
- update/delete ข้อมูล

- push, merge หรือสร้าง pull request
- deploy
- ส่งข้อความหรืออีเมล

Prompt สำหรับทำงานแบบปลอดภัย

text



ทำ P0 ที่พร้อมทำหนึ่งงานจาก TASKS.md

ก่อนเริ่ม:

1. แสดง task ที่เลือก
2. แสดงเหตุผลที่เลือก
3. แสดงไฟล์หรือระบบที่จะเปลี่ยน
4. แยกการกระทำเป็น safe actions และ approval-required actions

ทำ safe actions ได้ทันที

แต่ถ้าต้องแก้ Supabase, migration, push code, deploy, ลบไฟล์ หรือเปลี่ยนข้อมูลจริง ให้หยุดและขอ

approval

เมื่อเสร็จ ให้อัปเดต TASKS.md พร้อม
ผลลัพธ์และ blocker

จุดสำคัญคือให้ Agent **claim** งานก่อนเริ่ม และอัปเดตสถานะเมื่อเสร็จหรือถูก block ไม่เช่นนั้นการรันหลาย session อาจทำงานซ้ำกันได้ครับ.

การอ้างอิง:

[1] TASKS.md — A task queue for AI agents

<https://tasksmd.github.io/tasks.md/>

[2] Proposal: TASKS.md as a companion

standard for agent ... - GitHub [https://](https://github.com/agentsmd/agents.md/issues/166)

github.com/agentsmd/agents.md/issues/166

[3] nicobailon/pi-coordination [https://](https://github.com/nicobailon/pi-coordination)

github.com/nicobailon/pi-coordination

[4] Introduction - tasks [https://agenttasks.dev/](https://agenttasks.dev/docs)
docs

[5] TASKS.md - Task Decomposition |

specstoryai/specflow | DeepWiki <https://deepwiki.com/specstoryai/specflow/3.1.3-tasks.md-task-decomposition>

[6] agent-autonomy-kit skill by openclaw/skills - playbooks <https://playbooks.com/skills/openclaw/skills/agent-autonomy-kit>

[7] CLI Reference – Taskmarket <https://docs.taskmarket.dev/reference/cli>

[8] Task Management System | yushui2022/easy-coding-agents | DeepWiki <https://deepwiki.com/yushui2022/easy-coding-agents/6-task-management-system>

[9] Permissions, Sandboxes, and Autonomous Agents <https://paulgp.substack.com/p/permissions-sandboxes-and-autonomous>

[10] task-based-multiagent - Skill <https://smithery.ai/skills/melodic-software/task-based-multiagent>

[11] Claude Code Task Management: Native Multi-Session AI <https://claudefa.st/blog/guide/>

development/task-management

[12] Shared Task Queue (Cross-Worktree) - LLM

Wiki <https://vietbui.mintlify.app/concepts/>

shared-task-queue

สรุปการตั้งค่า Agent

คุณสามารถใช้ FIG Agent จัดการงานโดยไม่ต้องมี Jira หรือ Asana ได้ โดยใช้ TASKS.md เป็นแหล่งข้อมูลหลัก

โครงสร้างหลัก

markdown



Tasks

P0

P1

P2

P3

Waiting On

Done

- P0 – เร่งด่วนและสำคัญที่สุด
- P1 – สำคัญ ควรทำเร็ว
- P2 – ทำภายในสัปดาห์หรือเมื่อมีเวลา
- P3 – ยังไม่เร่งด่วน
- Waiting On – รอ approval ข้อมูล หรือบุคคลอื่น
- Done – งานเสร็จแล้ว

กฎที่ควรใส่ใน Knowledge

ใช้ TASKS.md เป็น source of truth สำหรับ task ทั้งหมด

เมื่อได้รับคำสั่ง "ทำงานต่อ":

1. อ่าน TASKS.md
2. เลือก P0 ที่ยังไม่เสร็จและไม่ blocked
3. ถ้าไม่มี ให้เลือก P1, P2 และ P3 ตามลำดับ
4. ตรวจสอบ dependency และ deadline
5. Claim task ก่อนเริ่มด้วย (@fig-agent)
6. ทำเฉพาะ task ที่ claim แล้ว
7. ทดสอบหรือ validate ผลลัพธ์
8. เมื่อเสร็จ ให้อัปเดต TASKS.md และย้ายไป Done
9. ถ้าติด blocker ให้ย้ายไป Waiting On พร้อมเหตุผล
10. หยุดและขอ approval ก่อนแก้ database, migration, deploy, push code, ลบไฟล์ หรือเปลี่ยนข้อมูลจริง

Prompt ใช้งานประจำ

text



ทำงานต่อจาก TASKS.md

เลือก task ที่พร้อมทำ โดยให้ P0 มาก่อน
ตรวจ blocker และ dependency ก่อนเริ่ม
claim งานด้วย (@fig-agent)
ทำงานให้เสร็จ ทดสอบผลลัพธ์ และอัปเดต
TASKS.md
หยุดเมื่อไม่มีงานที่พร้อมทำหรือมี action ที่
ต้องขอ approval

Workflow ที่แนะนำ

1. ใช้ daily-briefing เพื่อดูงานวันนี้

2. ใช้ task-management เพื่อเพิ่มและอัปเดตงาน
3. ใช้ prompt “ทำงานต่อ” เพื่อให้ Agent หยุด P0
4. ใช้ standup เพื่อสรุป เมื่อวาน / วันนี้ / Blockers
5. ใช้ Supabase schema inspection ก่อนทุกงานที่เกี่ยวข้องกับฐานข้อมูล

สิ่งสำคัญที่สุดคือให้ Agent อ่านก่อนเลือก, claim ก่อนทำ, ตรวจสอบก่อนปิดงาน และขอ approval ก่อนทำงานที่มีผลกระทบจริง ครับ.



Perplexity
www.perplexity.ai

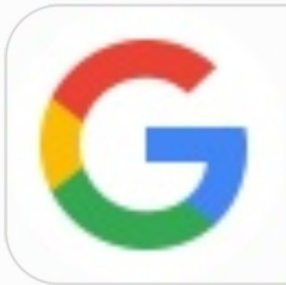
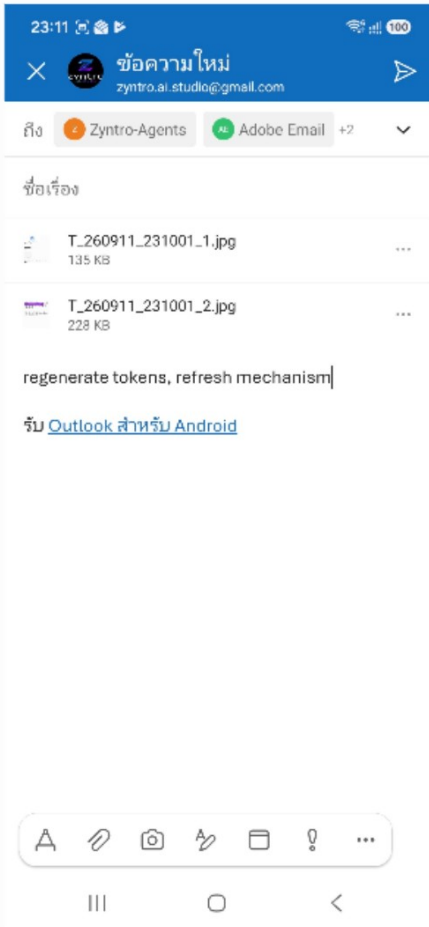


สมัครใช้ Fig Super Monthly \$20



Legal - Licensed Application End...
www.apple.com





Google Search
share.google



.



..



Legal - Licensed Application End...
www.apple.com

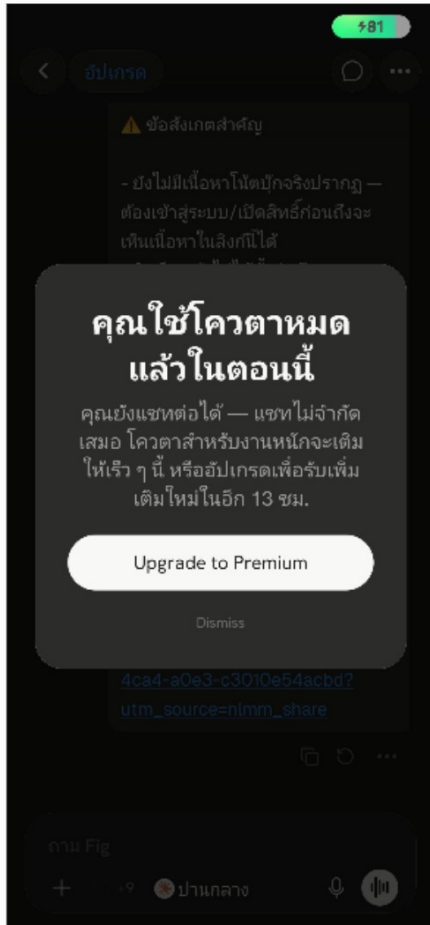


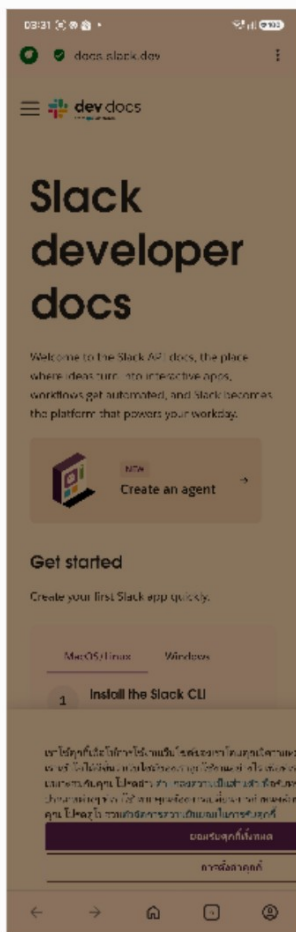


Legal - Licensed Application End...
www.apple.com



.





Done. Status:

Shipped — PR #206 (open, mergeable, 11 files, +1292/-10) and PR #207 (changelog). The Obsidian Local REST API bridge is in: full documented surface (vault read/write/append/patch/delete, active, JsonLogic + simple search, tags, commands, open), reads open, writes behind JWT and OBSIDIAN_ALLOW_WRITE=1, traversal rejected with 422, every payload through the CWE-1321 sanitizer. 33 tests pass — verified on a fresh clone of the pushed branch,

not just locally.

The workflow couldn't be pushed. The App still lacks workflows permission, so `.github/workflows/cwe1321-protection-suite.yml` was rejected. It's in the handoff bundle above — drop in `.yml` plus a `git am`-able patch that `git apply --check` is clean against fresh main, with all three actions pinned to full SHAs.

Two things worth knowing:

The red CI on PR #206 is not from my diff — it's a pre-existing repo policy failure. The repo requires full-SHA pins, and `ci.yml`, `test-suite.yml`, `secret-scan.yml`, `release_drafter.yml`, `dependabot-automerge.yml` and others still use `@v4/@v5`, so they die at job setup. Main is red for the same reason. My workflow is already pinned, so it will pass where those don't — but nothing will run my gate until the workflow lands. The "CWE-1321 suite" from the earlier thread had already merged; what was actually missing

was the CI wiring, which is what this closes.
Want me to open a follow-up PR pinning the existing workflows to SHAs? That's the fix that turns the board green, and I'd do it repo-wide in one pass rather than file by file.

สำเร็จ

 Full Complete Bundle — CrystalCastle (ID: 9nblggh69m0b)

Direct Link: <https://apps.microsoft.com/store/detail/crystalcastle/9nblggh69m0b>

 All Major Languages (Ready)

 Thai

markdown

```
<a href="https://apps.microsoft.com/store/detail/crystalcastle/9nblggh69m0b">
```

```
<picture>
```

```
<source media="(prefers-color-scheme: dark)"
```

```
srcset="https://assets.windowsphone.com/2621082c-4963-4fe1-85d3-7fccdfefde99/
```

```
Thai_get_Dark.ashx">
```

```

```

```
</picture>
```

```
</a>
```

 English

markdown

```
<a href="https://apps.microsoft.com/store/detail/crystalcastle/9nblggh69m0b">
```

```
<picture>
```

```
<source media="(prefers-color-scheme: dark)"
srcset="https://assets.windowsphone.com/
2621082c-4963-4fe1-85d3-7fccdfefde99/
English_get_Dark.ashx">
```

```

```

```
</picture>
```

```
</a>
```

 Chinese (Simplified)

markdown

```
<a href="https://apps.microsoft.com/store/
```

```
detail/crystalcastle/9nblggh69m0b">
<picture>
<source media="(prefers-color-scheme: dark)"
srcset="https://assets.windowsphone.com/
2621082c-4963-4fe1-85d3-7fccdfefde99/
ChineseSimplified_get_Dark.ashx">

</picture>
</a>
```

 Japanese

markdown

```
<a href="https://apps.microsoft.com/store/
detail/crystalcastle/9nblggh69m0b">
<picture>
```

```
<source media="(prefers-color-scheme: dark)"
srcset="https://assets.windowsphone.com/
2621082c-4963-4fe1-85d3-7fccdfefde99/
Japanese_get_Dark.ashx">

</picture>
</a>
```

 Spanish

markdown

```
<a href="https://apps.microsoft.com/store/
detail/crystalcastle/9nblggh69m0b">
<picture>
<source media="(prefers-color-scheme: dark)"
srcset="https://assets.windowsphone.com/
```

```
2621082c-4963-4fe1-85d3-7fccdfefde99/  
Spanish_get_Dark.ashx">  
  
</picture>  
</a>
```

 Full Size Set (All Heights)

Small (40px)

markdown

```
<a href="https://apps.microsoft.com/store/  
detail/crystalcastle/9nblggh69m0b">  
  
</a>
```

Medium (52px — Default)

markdown

```
<a href="https://apps.microsoft.com/store/  
detail/crystalcastle/9nblggh69m0b">  
  
</a>
```

Large (80px)

markdown

```
<a href="https://apps.microsoft.com/store/detail/crystalcastle/9nblggh69m0b">
```

```

```

```
</a>
```

✨ Custom Branded SVG (CrystalCastle Logo + Style)

svg

```
<svg width="260" height="64" viewBox="0 0 260 64" fill="none" xmlns="http://www.w3.org/2000/svg">
```

```
<defs>
```



```
<linearGradient id="grad" x1="0" y1="0"
x2="100%" y2="100%">
<stop offset="0%" stop-color="#4F46E5"/>
<stop offset="100%" stop-color="#06B6D4"/>
</linearGradient>
</defs>
<rect width="260" height="64" rx="12"
fill="url(#grad)"/>
<path d="M32 20L40 28V44H24V28L32 20Z"
fill="white" opacity="0.9"/>
<path d="M40 28L48 36V44H32V36L40 28Z"
fill="white" opacity="0.7"/>
<text x="72" y="32" font-family="Segoe UI,
Roboto" font-size="14" font-weight="600"
fill="white">CrystalCastle</text>
<text x="72" y="48" font-family="Segoe UI" font-
size="11" fill="white" opacity="0.9">Available on
Microsoft Store</text>
</svg>
```

Auto-Detect JS Component

html

```
<div class="ms-store-button"  
data-product-id="9nblggh69m0b"  
data-size="medium"  
data-theme="auto"  
data-locale="auto">  
</div>  
  
<script async src="https://cdn.microsoft.com/  
store/api/button/v1/init.js"></script>
```

React + TypeScript (Multi-Lang Ready)

tsx


```
import React from 'react';
```

```
type Locale = 'en' | 'th' | 'zh' | 'ja' | 'es';
```

```
type Size = 'small' | 'medium' | 'large';
```

```
interface Props {
```

```
  locale?: Locale;
```

```
  size?: Size;
```

```
}
```

```
const MicrosoftStoreBadge: React.FC<Props> =
```

```
({
```

```
  locale = 'en',
```

```
  size = 'medium'
```

```
}) => {
```

```
  const heights = { small: 40, medium: 52, large:  
    80 };
```

```
  const alts = {
```

```
    en: 'Get CrystalCastle from Microsoft Store',
```

```
    th: 'รับ CrystalCastle จาก Microsoft Store',
```

```
zh: '从 Microsoft Store 获取 CrystalCastle',  
ja: 'Microsoft Store から CrystalCastle を入手',  
es: 'Obtener CrystalCastle en Microsoft Store'  
};
```

```
return (
```

```
<a  
href="https://apps.microsoft.com/store/detail/  
crystalcastle/9nblggh69m0b"  
target="_blank"  
rel="noopener noreferrer"  
>
```

```
<picture>
```

```
<source media="(prefers-color-scheme: dark)"  
srcSet={https://assets.windowsphone.com/  
2621082c-4963-4fe1-85d3-7fccdfefde99/$  
{locale}_get_Dark.ashx} /> <img  
src={https://assets.windowsphone.com/  
85862733-836c-4600-93a5-25b30c47b198/$  
{locale}_get_Light.ashx}
```

```
alt={alts[locale]}  
height={heights[size]}  
/>  
</picture>  
</a>  
);  
};
```

```
export default MicrosoftStoreBadge;
```

✅ Everything ready: 5 languages • 3 sizes • custom SVG • React/JS/HTML • Dark Mode
You can copy/paste directly into README, website, docs, or app.

Want me to combine all into one ready-to-save README-badges.md file?